

1 What does gradient descent update in this model?

This section explains (at a practical implementation level) what parameters are updated by gradient-based optimization when training the models in this repository, focusing on the proposed **EGMN**.

1.1 What parameters are learnable?

In PyTorch, **gradient descent updates all tensors registered as parameters** (i.e. returned by `model.parameters()`). Concretely for EGMN, these include:

- **Embedding tables** for sparse and sequence features.
- **Linear layers** for continuous features (e.g. `duration` projection).
- **Shared MLP (trunk)** weights and biases.
- **Output heads** predicting the mixture parameters:
 - mixture logits π (pre-softmax),
 - exponential rate λ (via `softplus`),
 - Gaussian parameters μ and σ (via `softplus` in the implementation).

1.2 What is the objective being minimized?

During training, gradients are computed for a scalar loss. For EGMN, the training script forms:

$$\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha \mathcal{L}_{\text{entropy}} + \beta \mathcal{L}_{\text{reg}}.$$

- $\mathcal{L}_{\text{NLL}}$: negative log-likelihood of the observed watch time under the mixture distribution.
- $\mathcal{L}_{\text{reg}}$: reconstruction-style regularizer (L1 loss) between the observed label and the mixture-mean prediction.
- $\mathcal{L}_{\text{entropy}}$: a mixture entropy term computed from the mixture weights (controls component usage), weighted by α .

1.3 What does “gradient descent” do here?

At each iteration, PyTorch computes gradients $\nabla_{\theta} \mathcal{L}$ for all learnable parameters θ . The optimizer (e.g. Adagrad/Adam) then applies an update of the form:

$$\theta \leftarrow \theta - \eta \Delta(\nabla_{\theta} \mathcal{L}),$$

where η is the learning rate and $\Delta(\cdot)$ depends on the optimizer (e.g. per-parameter adaptive scaling).

1.4 Code example: the update step in training

The training scripts in this repo follow the standard PyTorch pattern:

```
# run_egmn.py / run_*.py (illustrative)
optimizer = torch.optim.Adagrad(model.parameters(), lr=args.lr,
                                weight_decay=args.weight_decay)

for features, label in dataloaders["train"]:
    # Forward: compute distribution parameters
    pi, lambda_, mu, sigma = model(features)

    # Loss: compute scalar objective
    nll, reg, ent = model.loss(label.float(), pi, lambda_, mu, sigma,
                               features["duration"].view(-1, 1))
    loss = nll + args.alpha * ent + args.beta * reg

    # Backward: accumulate gradients d(loss)/d(theta)
    optimizer.zero_grad()
    loss.backward()

    # Update: apply optimizer rule to parameters theta
    optimizer.step()
```

1.5 How gradients flow to mixture parameters

Even though EGMN conceptually outputs π , λ , μ , σ , these are *not* parameters themselves; they are **functions of the network weights**. Gradients flow from the loss through these outputs back into the head layers and the shared trunk.

```
# model/egmn.py (illustrative)
# pi is produced by a linear layer and normalized via softmax
+pi = self.mixture_logits(hidden) # logits
+mix_probs = torch.softmax(pi, dim=1) # weights

# lambda, mu, sigma are produced by heads with softplus constraints
+lambda_ = self.lambda_layer(hidden) + 1e-6
+mu = self.gauss_mu(hidden) + 1 / lambda_
+sigma = self.gauss_sigma(hidden) + 1e-6

# The NLL depends on (mix_probs, lambda_, mu, sigma),
# so gradients propagate into all head/trunk parameters.
```

1.6 Which parts are (not) updated?

- Updated: any parameters owned by the model modules passed to the optimizer.
- Not updated: input tensors (features/labels), data preprocessing artifacts (e.g. bucket boundaries / quantile tables), and any values created under `torch.no_grad()`.